

Estudo Dirigido

Introdução ao Pygame

Configuração Inicial e Criação de Telas

Este estudo dirigido fornece uma introdução prática à biblioteca Pygame, cobrindo os passos essenciais para a configuração de um projeto do zero: inicialização, criação da janela e implementação do loop principal do jogo.

Objetivos de Aprendizagem

- Entender a estrutura básica de um projeto Pygame
- Inicializar os módulos do Pygame corretamente
- Criar e configurar a janela do jogo
- Implementar o Loop Principal (Game Loop)
- Compreender o ciclo de Eventos → Atualização → Renderização

1. O que é o Pygame?

Pygame é um conjunto de módulos Python construído sobre a biblioteca SDL (Simple DirectMedia Layer). Ele fornece ferramentas de alto nível para criação de jogos 2D, incluindo gerenciamento de janelas, desenho de gráficos, reprodução de sons e captura de eventos de teclado e mouse.

Vantagens	Casos de Uso Ideais
- Simples de aprender com Python - Documentação extensa e comunidade ativa - Multiplataforma (Windows, macOS, Linux)	- Protótipos e jogos 2D - Aprendizado de lógica de jogos - Projetos educacionais e acadêmicos

2. Configuração Inicial

2.1. Instalação

Antes de começar, instale o Pygame via pip no terminal:

```
# No terminal / prompt de comando
```

```
pip install pygame
```

💡 Verificando a instalação

Execute `python -m pygame --version` no terminal. Se a versão for exibida, a instalação foi bem-sucedida.

2.2. Inicialização do Pygame

O primeiro passo em qualquer projeto Pygame é inicializar todos os módulos com uma única chamada. Essa função carrega os subsistemas de gráficos, som, fontes, entre outros.

```
import pygame
import sys

# Inicializa TODOS os módulos do Pygame
# Retorna (sucesso, falha) — ideal verificar em projetos maiores
pygame.init()
```

💡 Boa prática

Em projetos maiores, verifique o retorno: `sucesso, falha = pygame.init()`. Se `falha > 0`, algum módulo não carregou corretamente.

2.3. Definição e Criação da Tela

Após a inicialização, defina as dimensões da janela e crie o objeto de superfície da tela. Todas as imagens e formas serão desenhadas nessa superfície.

Variável	Descrição
<code>SCREEN_WIDTH</code>	Largura da tela em pixels (ex: 800)
<code>SCREEN_HEIGHT</code>	Altura da tela em pixels (ex: 600)
<code>SCREEN</code>	Objeto Surface — onde todo conteúdo é desenhado

```
# Dimensões da tela
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600

# Cria a janela do jogo (Surface)
SCREEN = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))

# Define o título da barra de título
pygame.display.set_caption("Meu Primeiro Jogo Pygame")
```

💡 Flags de display

Você pode passar flags opcionais: `pygame.FULLSCREEN` para tela cheia, `pygame.RESIZABLE` para janela redimensionável, ou `pygame.NOFRAME` para janela sem bordas.

3. O Loop Principal do Jogo

Todo jogo em tempo real funciona dentro de um Loop Principal (Game Loop). Esse loop executa continuamente enquanto o jogo estiver rodando e é responsável por três etapas fundamentais:

1. Eventos	2. Atualização	3. Renderização
Captura entradas do usuário e do sistema	Move objetos, calcula colisões, atualiza pontuação	Desenha todos os elementos na tela

3.1. Implementação Completa

O código abaixo reúne todos os conceitos apresentados em um esqueleto funcional:

```
import pygame
import sys

# — Inicialização —
pygame.init()

# — Configuração da Tela —
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
SCREEN = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Meu Primeiro Jogo Pygame")

# — Cores (R, G, B) —
PRETO = ( 0, 0, 0)
BRANCO = (255, 255, 255)
AZUL = ( 30, 100, 200)

# — Controle de FPS —
clock = pygame.time.Clock()
FPS = 60
running = True

# — Game Loop —
while running:
```

```
# 1. Manipulação de Eventos
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            running = False

# 2. Atualização do Estado (lógica do jogo)
# TODO: adicionar movimentação, colisões, etc.

# 3. Renderização
SCREEN.fill(PRETO)          # limpa o frame anterior
pygame.display.flip()       # exhibe o frame na tela

# Limita a 60 quadros por segundo
clock.tick(FPS)

# — Encerramento —
pygame.quit()
sys.exit()
```

3.2. Funções-Chave do Loop

Variável	Descrição
<code>pygame.event.get()</code>	Retorna a fila de eventos pendentes (clique, tecla, sair...)
<code>SCREEN.fill(cor)</code>	Preenche toda a tela com uma cor, apagando o frame anterior
<code>pygame.display.flip()</code>	Atualiza a janela exibindo tudo que foi desenhado no frame
<code>clock.tick(FPS)</code>	Controla a velocidade do loop, limitando os quadros por segundo

4. Conceitos Importantes

4.1. Sistema de Coordenadas

No Pygame, o ponto (0, 0) fica no canto superior esquerdo da janela. O eixo X cresce para a direita e o eixo Y cresce para baixo — o oposto do plano cartesiano matemático tradicional.

Exemplo de posições (800×600):

- (0, 0) → canto superior esquerdo
- (800, 0) → canto superior direito
- (0, 600) → canto inferior esquerdo
- (400, 300) → centro da tela

Desenhando um retângulo:

```
# (surface, cor, (x, y, w, h))
pygame.draw.rect(
    SCREEN, AZUL,
    (100, 100, 200, 150)
)
```

4.2. Cores em RGB

Todas as cores no Pygame são definidas como tuplas com três valores inteiros de 0 a 255, representando os canais Vermelho (R), Verde (G) e Azul (B).

```
# Cores básicas
PRETO = ( 0, 0, 0)
BRANCO = (255, 255, 255)
VERMELHO = (255, 0, 0)
VERDE = ( 0, 255, 0)
AZUL = ( 0, 0, 255)
AMARELO = (255, 255, 0)
CINZA = (128, 128, 128)
```

5. Exercícios Práticos

Coloque em prática os conceitos aprendidos com os exercícios a seguir:

Exercício 1 — Janela colorida

Modifique o código para que a tela tenha fundo azul (0, 100, 200) em vez de preto.

Exercício 2 — Título personalizado

Altere o título da janela para exibir o seu nome e o texto 'Primeiro Jogo'.

Exercício 3 — Resolução diferente

Crie uma janela com resolução de 1024×768 pixels e verifique as diferenças visuais.

Exercício 4 — FPS variável

Experimente valores diferentes para FPS (30, 60, 120) e observe o comportamento do loop.

Exercício 5 — Tecla de saída

Adicione suporte para fechar o jogo ao pressionar a tecla ESC (já presente no código-exemplo).

6. Referência Rápida

Variável	Descrição
<code>pygame.init()</code>	Inicializa todos os módulos do Pygame
<code>pygame.display.set_mode()</code>	Cria a janela do jogo e retorna a Surface principal
<code>pygame.display.set_caption()</code>	Define o texto exibido na barra de título
<code>pygame.event.get()</code>	Retorna todos os eventos pendentes na fila
<code>pygame.QUIT</code>	Evento disparado ao fechar a janela
<code>pygame.KEYDOWN</code>	Evento disparado ao pressionar uma tecla
<code>Surface.fill(cor)</code>	Preenche a superfície com uma cor sólida
<code>pygame.display.flip()</code>	Atualiza toda a janela (double buffering)
<code>pygame.time.Clock()</code>	Cria um objeto para controle de FPS
<code>clock.tick(fps)</code>	Limita o loop à taxa de quadros especificada
<code>pygame.quit()</code>	Descarrega todos os módulos do Pygame
<code>sys.exit()</code>	Encerra o processo Python completamente